

Probabilistic Reasoning in Computer Science

Milan Češka



MSP 2025

Outline

Part 1: Modelling and Analysis of Probabilistic Systems

- Lecture 1: Markov Chains
- Lecture 2: Markov Decision Processes

These lectures are partially based on material from Chapter 10 of "Principles of Model Checking" by Christel Baier and Joost-Pieter Katoen (MIT Press) and "Stochastic Model Checking" by Marta Kwiatkowska, Gethin Norman and David Parker. The slides use material from <https://www.prismmodelchecker.org/lectures/pmc/>

Part 2: Introduction to Randomised Algorithms

- Lecture 3: Probabilistic analysis of programs and basic randomised algorithms

This lecture is partially based on material from Chapter 5 of “**Introduction to Algorithms (3rd edition)**” by Thomas Cormen et al. (MIT Press).

Probabilistic Analysis of Programs

Goes beyond the best case or worst case analysis

- analysis of all cases (inputs) using their probability distribution
- in many cases provide a better insight into the practical complexity

Probabilistic Analysis of Programs

Goes beyond the best case or worst case analysis

- analysis of all cases (inputs) using their probability distribution
- in many cases provide a better insight into the practical complexity

Example (The Hiring Problem)

Hire-Assistant(n)

1. `best = 0`
2. `for i from 1 to n:`
3. `interview candidate i`
4. `if candidate i is better than best:`
5. `hire candidate i` ▷ paying cost c_h
6. `best = i`

Probabilistic Analysis of Programs

Goes beyond the best case or worst case analysis

- analysis of all cases (inputs) using their probability distribution
- in many cases provide a better insight into the practical complexity

Example (The Hiring Problem)

Hire-Assistant(n)

1. `best = 0`
2. `for i from 1 to n:`
3. `interview candidate i`
4. `if candidate i is better than best:`
5. `hire candidate i` $\triangleright$ paying cost c_h
6. `best = i`

Non-probabilistic analysis for n candidates

- worst-case $\mathcal{O}(n \cdot c_h)$
- best-case $\mathcal{O}(c_h)$

Probabilistic Analysis of Programs

Hire-Assistant(n)

```
1. best = 0
2. for i from 1 to n:
3.     interview candidate i
4.     if candidate i is better than best:
5.         hire candidate i      ▷ paying cost  $c_h$ 
6.         best = i
```

Probabilistic analysis for n candidates

- what do we know about the inputs (candidates) distribution?
- for now assume **a random** order of the candidates — we will get back to this later
- we have $n!$ permutations every equally likely
- how many permutations require 1,2, ..., n hirings — too complicated
- **compute expected number of hirings using indicator random variable**

Indicator Random Variable

- given a sample space S and an event A in S
- indicator random variable for A is defined as $I\{A\} = 1$ if A occurs, 0 otherwise
- let $X_A = I\{A\}$, it holds that $E[X_A] = Pr\{A\}$ (probability of A)

Indicator Random Variable

- given a sample space S and an event A in S
- indicator random variable for A is defined as $I\{A\} = 1$ if A occurs, 0 otherwise
- let $X_A = I\{A\}$, it holds that $E[X_A] = Pr\{A\}$ (probability of A)

Example

- What is expected number of heads (H) in n coin flips?

Indicator Random Variable

- given a sample space S and an event A in S
- indicator random variable for A is defined as $I\{A\} = 1$ if A occurs, 0 otherwise
- let $X_A = I\{A\}$, it holds that $E[X_A] = \Pr\{A\}$ (probability of A)

Example

- What is expected number of heads (H) in n coin flips?
- $X_i = I\{\text{i-th coin flip results in event H}\}$ where $\Pr\{X_i\} = E[X_i] = 1/2$
- $X = \sum_{i=1}^n X_i$
- $E[X] = E[\sum_{i=1}^n X_i]$ due to linearity we obtain $E[X] = \sum_{i=1}^n E[X_i] = n/2$

Indicator Random Variable

- given a sample space S and an event A in S
- indicator random variable for A is defined as $I\{A\} = 1$ if A occurs, 0 otherwise
- let $X_A = I\{A\}$, it holds that $E[X_A] = Pr\{A\}$ (probability of A)

Example

- What is expected number of heads (H) in n coin flips?
- $X_i = I\{\text{i-th coin flip results in event H}\}$ where $Pr\{X_i\} = E[X_i] = 1/2$
- $X = \sum_{i=1}^n X_i$
- $E[X] = E[\sum_{i=1}^n X_i]$ due to linearity we obtain $E[X] = \sum_{i=1}^n E[X_i] = n/2$

The key idea: Express the expectation of a random variable (X) as the expectation of the sum of components that are easier to count (the indicator variables X_i)

Analysis Using Indicator Random Variable

Hiring problem — we assume random order of the candidates

- define indicator variables $X_i = I\{\text{i-th candidate is hired}\}$
- $X = \sum_{i=1}^n X_i$ (the total number of hired candidate — the cost)
- $E[X]$ is the expected cost of the algorithm
- $E[X_i]$ is the probability that i-th candidate is hired

Analysis Using Indicator Random Variable

Hiring problem — **we assume random order of the candidates**

- define indicator variables $X_i = I\{\text{i-th candidate is hired}\}$
- $X = \sum_{i=1}^n X_i$ (the total number of hired candidate — the cost)
- $E[X]$ is the expected cost of the algorithm
- $E[X_i]$ is the probability that i-th candidate is hired
- candidate i is hired if it is better than candidates 1,2, ..., i-1
- $E[X_i] = 1/i$
- $E[X] = E[\sum_{i=1}^n X_i] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n 1/i = \ln n + \mathcal{O}(1)$

Analysis Using Indicator Random Variable

Hiring problem — we assume random order of the candidates

- define indicator variables $X_i = I\{\text{i-th candidate is hired}\}$
- $X = \sum_{i=1}^n X_i$ (the total number of hired candidate — the cost)
- $E[X]$ is the expected cost of the algorithm
- $E[X_i]$ is the probability that i-th candidate is hired
- candidate i is hired if it is better than candidates 1,2, ..., i-1
- $E[X_i] = 1/i$
- $E[X] = E[\sum_{i=1}^n X_i] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n 1/i = \ln n + \mathcal{O}(1)$

The analysis of the average complexity gives us an asymptotically better bound

- $\mathcal{O}(\log n \cdot c_h)$ vs $\mathcal{O}(n \cdot c_h)$

Randomised Algorithms

How to handle “adversarial” inputs (i.e. achieved the expected complexity for all inputs)?

Randomised Algorithms

How to handle “adversarial” inputs (i.e. achieved the expected complexity for all inputs)?

Randomisation

Randomised Algorithms

How to handle “adversarial” inputs (i.e. achieved the expected complexity for all inputs)?

Randomisation

In general, there are two approaches to randomisation

1. randomise the ordering of the inputs
2. randomise choices made within the algorithm

Randomised Algorithms

How to handle “adversarial” inputs (i.e. achieved the expected complexity for all inputs)?

Randomisation

In general, there are two approaches to randomisation

1. randomise the ordering of the inputs
2. randomise choices made within the algorithm

How to randomise in the hiring problem?

Randomised Algorithms

How to handle “adversarial” inputs (i.e. achieved the expected complexity for all inputs)?

Randomisation

In general, there are two approaches to randomisation

1. randomise the ordering of the inputs
2. randomise choices made within the algorithm

How to randomise in the hiring problem?

- randomise the list of candidates before the algorithm

Randomised Algorithms

How to handle “adversarial” inputs (i.e. achieved the expected complexity for all inputs)?

Randomisation

In general, there are two approaches to randomisation

1. randomise the ordering of the inputs
2. randomise choices made within the algorithm

How to randomise in the hiring problem?

- randomise the list of candidates before the algorithm

Famous example of the second approach?

Randomised Algorithms

How to handle “adversarial” inputs (i.e. achieved the expected complexity for all inputs)?

Randomisation

In general, there are two approaches to randomisation

1. randomise the ordering of the inputs
2. randomise choices made within the algorithm

How to randomise in the hiring problem?

- randomise the list of candidates before the algorithm

Famous example of the second approach?

- Quick Sort (see later)

Classification of Randomised Algorithms

So far, we gambled with resources (i.e. with the run-time).

Classification of Randomised Algorithms

So far, we gambled with resources (i.e. with the run-time).

We can also gamble with correctness

Classification of Randomised Algorithms

So far, we gambled with resources (i.e. with the run-time).

We can also gamble with correctness

2 types of random algorithms

1. Las Vegas algorithms

- for every input, give correct results (correctness is guaranteed)
- for every input, there is a probability that the running time is larger than expected/desired
- e.g. the hiring algorithm or Quick Sort

2. Monte Carlo algorithms

- for every input, there is a probability of error (incorrect result)
- the worst-case running time is guaranteed

Classification of Randomised Algorithms

Simple example: *Given an array $A[1 \dots n]$ with n elements (n even). Half of the elements contains 0s, the other half contains 1s. Goal: Find an index that contains a 1.*

Construct Las Vegas and Monte Carlo algorithms

- should (in some aspects) outperform the correct $\mathcal{O}(n)$ deterministic algorithm

Classification of Randomised Algorithms

Simple example: *Given an array $A[1 \dots n]$ with n elements (n even). Half of the elements contains 0s, the other half contains 1s. Goal: Find an index that contains a 1.*

Construct Las Vegas and Monte Carlo algorithms

- should (in some aspects) outperform the correct $\mathcal{O}(n)$ deterministic algorithm
- Monte Carlo with time $\mathcal{O}(1)$
- Las Vegas with expected time $\mathcal{O}(1)$

Classification of Randomised Algorithms

Simple example: *Given an array $A[1 \dots n]$ with n elements (n even). Half of the elements contains 0s, the other half contains 1s. Goal: Find an index that contains a 1.*

Construct Las Vegas and Monte Carlo algorithms

- should (in some aspects) outperform the correct $\mathcal{O}(n)$ deterministic algorithm

repeat:

$k = \text{RandInt}(n)$

if $A[k] = 1$, **return** k

repeat 300 times:

$k = \text{RandInt}(n)$

if $A[k] = 1$, **return** k

return “Failed”

- Las Vegas with expected time $\mathcal{O}(1)$
- Monte Carlo with time $\mathcal{O}(1)$

Classification of Randomised Algorithms

Simple example: *Given an array $A[1 \dots n]$ with n elements (n even). Half of the elements contains 0s, the other half contains 1s. Goal: Find an index that contains a 1.*

Construct Las Vegas and Monte Carlo algorithms

- should (in some aspects) outperform the correct $\mathcal{O}(n)$ deterministic algorithm

```
repeat:  
  k = RandInt(n)  
  if A[k] = 1, return k
```

```
repeat 300 times:  
  k = RandInt(n)  
  if A[k] = 1, return k  
return "Failed"
```

amplification
improves the
probability

- Las Vegas with expected time $\mathcal{O}(1)$
- Monte Carlo with time $\mathcal{O}(1)$

Classification of Randomised Algorithms

Simple example: *Given an array $A[1 \dots n]$ with n elements (n even). Half of the elements contains 0s, the other half contains 1s. Goal: Find an index that contains a 1.*

```
repeat:  
    k = RandInt(n)  
    if A[k] = 1, return k
```

- How we can prove the expected runtime of this algorithm?
- Use the indicator variable

Classification of Randomised Algorithms

Simple example: *Given an array $A[1 \dots n]$ with n elements (n even). Half of the elements contains 0s, the other half contains 1s. Goal: Find an index that contains a 1.*

repeat:
 $k = \text{RandInt}(n)$
 if $A[k] = 1$, **return** k

- How we can prove the expected runtime of this algorithm?
- Use the indicator variable
- $X_i = 1$ if the algorithm performs the i -th comparison, 0 otherwise
- the complexity is given as $X = \sum_i^\infty X_i$
- $E[X] = E[\sum_i^\infty X_i] = \sum_i^\infty E[X_i] = \sum_i^\infty \Pr\{X_i\} = \sum_i^\infty 1/2^{i-1} = 2$

Quick Sort

On input $S = (x_1, x_2, \dots, x_n)$

- If $n \leq 1$, return S
 - Pick uniformly at random a “pivot” x_m
 - Compare x_m to all other x 's
 - Let $S_1 = \{x_i : x_i < x_m\}$, $S_2 = \{x_i : x_i > x_m\}$
 - Recursively sort S_1 and S_2 .
 - Return $[S_1, x_m, S_2]$
-

Las Vegas algorithm

- always gives correct result
- the complexity (total number of comparisons) depends on the pivot

Quick Sort

On input $S = (x_1, x_2, \dots, x_n)$

- If $n \leq 1$, return S
 - Pick uniformly at random a “pivot” x_m
 - Compare x_m to all other x 's
 - Let $S_1 = \{x_i : x_i < x_m\}$, $S_2 = \{x_i : x_i > x_m\}$
 - Recursively sort S_1 and S_2 .
 - Return $[S_1, x_m, S_2]$
-

Las Vegas algorithm

- always gives correct result
- the complexity (total number of comparisons) depends on the pivot

$$T(n) = T(|S_1|) + T(|S_2|) + (n - 1)$$

$$T(n) = T(k) + T(n - k - 1) + (n - 1)$$

Quick Sort

On input $S = (x_1, x_2, \dots, x_n)$

- If $n \leq 1$, return S
- Pick uniformly at random a “pivot” x_m
- Compare x_m to all other x 's
- Let $S_1 = \{x_i : x_i < x_m\}$, $S_2 = \{x_i : x_i > x_m\}$
- Recursively sort S_1 and S_2 .
- Return $[S_1, x_m, S_2]$

Las Vegas algorithm

- always gives correct result
- the complexity (total number of comparisons) depends on the pivot

$$T(n) = T(|S_1|) + T(|S_2|) + (n - 1)$$

$$T(n) = T(k) + T(n - k - 1) + (n - 1)$$

recursive calls

before recursion

Quick Sort

On input $S = (x_1, x_2, \dots, x_n)$

- If $n \leq 1$, return S
- Pick uniformly at random a “pivot” x_m
- Compare x_m to all other x 's
- Let $S_1 = \{x_i : x_i < x_m\}$, $S_2 = \{x_i : x_i > x_m\}$
- Recursively sort S_1 and S_2 .
- Return $[S_1, x_m, S_2]$

$$T(n) = T(|S_1|) + T(|S_2|) + (n - 1)$$

$$T(n) = T(k) + T(n - k - 1) + (n - 1)$$

- worst case $k = 0$ leads to $\mathcal{O}(n^2)$
- best case $k \sim n/2$ leads to $\mathcal{O}(n \cdot \log(n))$

What is the expected case?

Quick Sort — Probabilistic Analysis

On input $S = (x_1, x_2, \dots, x_n)$

- If $n \leq 1$, return S
 - Pick uniformly at random a “pivot” x_m
 - Compare x_m to all other x 's
 - Let $S_1 = \{x_i : x_i < x_m\}$, $S_2 = \{x_i : x_i > x_m\}$
 - Recursively sort S_1 and S_2 .
 - Return $[S_1, x_m, S_2]$
-

How many times x_i and x_j get compared?

Quick Sort — Probabilistic Analysis

On input $S = (x_1, x_2, \dots, x_n)$

- If $n \leq 1$, return S
 - Pick uniformly at random a “pivot” x_m
 - Compare x_m to all other x 's
 - Let $S_1 = \{x_i : x_i < x_m\}$, $S_2 = \{x_i : x_i > x_m\}$
 - Recursively sort S_1 and S_2 .
 - Return $[S_1, x_m, S_2]$
-

How many times x_i and x_j get compared? **0-time or 1-time**

$$X_{ij} = \begin{cases} 1 & \text{if } x_i \text{ and } x_j \text{ are compared} \\ 0 & \text{otherwise} \end{cases}$$

Quick Sort — Probabilistic Analysis

On input $S = (x_1, x_2, \dots, x_n)$

- If $n \leq 1$, return S
 - Pick uniformly at random a “pivot” x_m
 - Compare x_m to all other x 's
 - Let $S_1 = \{x_i : x_i < x_m\}$, $S_2 = \{x_i : x_i > x_m\}$
 - Recursively sort S_1 and S_2 .
 - Return $[S_1, x_m, S_2]$
-

How many times x_i and x_j get compared? **0-time or 1-time**

$$X_{ij} = \begin{cases} 1 & \text{if } x_i \text{ and } x_j \text{ are compared} \\ 0 & \text{otherwise} \end{cases}$$

Total number of comparison $X = \sum_{1 \leq i < j \leq n} X_{ij}$

Quick Sort — Probabilistic Analysis

$$\mathbf{E}[X] = \sum_{1 \leq i < j \leq n} \mathbf{Pr}[x_i \text{ and } x_j \text{ are compared}]$$

Let $y_1, y_2, \dots, y_n$ be the input elements in sorter order.

x_1	x_2	x_3	x_4	x_5	x_6	x_7
4	8	2	7	99	5	0
y_3	y_6	y_2	y_5	y_7	y_4	y_1

$$\mathbf{E}[X] = \sum_{1 \leq i < j \leq n} \mathbf{Pr}[y_i \text{ and } y_j \text{ are compared}]$$

Quick Sort — Probabilistic Analysis

Claim: $\Pr[y_i \text{ and } y_j \text{ are compared}] = \frac{2}{j - i + 1}$

Proof: Define $Y^{ij} = \{y_i, y_{i+1}, \dots, y_j\}$

Consider the algorithm when a pivot p is being chosen.

Case 1: $p \notin Y^{ij} \implies Y^{ij} \subseteq S_1 \quad \text{or} \quad Y^{ij} \subseteq S_2$

Case 2: $p \in Y^{ij}$ but $p \neq y_i$ and $p \neq y_j$
 $\implies y_i \in S_1$ and $y_j \in S_2$ (y_i and y_j never compared)

Case 3: $p = y_i$ or $p = y_j$ (y_i and y_j are compared)

Quick Sort — Probabilistic Analysis

Claim: $\Pr[y_i \text{ and } y_j \text{ are compared}] = \frac{2}{j - i + 1}$

Proof: Define $Y^{ij} = \{y_i, y_{i+1}, \dots, y_j\}$

Conclusion: y_i and y_j get compared iff

y_i or y_j is chosen as pivot before $y_{i+1}, y_{i+2}, \dots, y_{j-1}$

i.e., the first pivot from Y^{ij} was y_i or y_j .

What is the probability of this? $\frac{2}{|Y^{ij}|} = \frac{2}{j - i + 1}$

Quick Sort — Probabilistic Analysis

Claim: $\Pr[y_i \text{ and } y_j \text{ are compared}] = \frac{2}{j - i + 1}$

$$\mathbf{E}[X] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1} = 2 \sum_{1 \leq i < j \leq n} \frac{1}{j - i + 1}$$

$$\begin{aligned} \sum_{1 \leq i < j \leq n} \frac{1}{j - i + 1} &= \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n-1} + \frac{1}{n} \quad (\leq \ln n) \\ &\quad + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n-1} \\ &\quad + \cdots \\ &\quad + \frac{1}{2} \end{aligned}$$

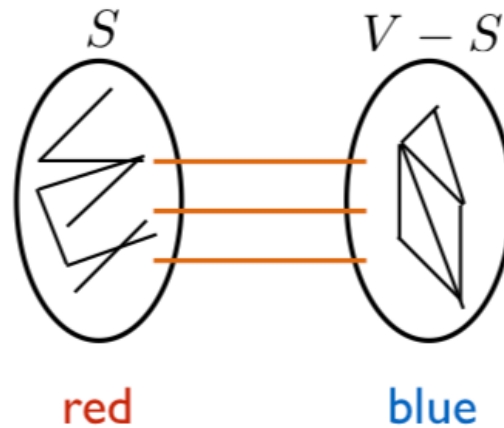
$$\begin{aligned} \text{So: } \mathbf{E}[X] &\leq 2n \ln n \\ &= O(n \log n) \end{aligned}$$

We obtained the expected complexity $\mathcal{O}(n \cdot \log(n))$

Min Cut

Min Cut Problem

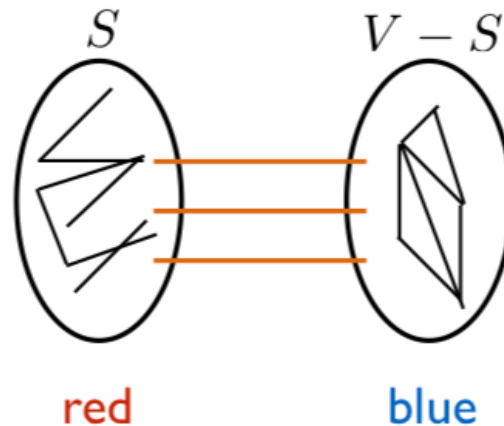
Given a graph $G = (V, E)$,
find a non-empty subset $S \subset V$ such that
number of edges from S to $V - S$ is minimized.



Min Cut

Min Cut Problem

Given a graph $G = (V, E)$,
find a non-empty subset $S \subset V$ such that
number of edges from S to $V - S$ is minimized.



There is 2^n cuts where $n = |S|$ — this gives us a naive algorithm

There is a very complex deterministic polynomial algorithm.

There is simple and more efficient randomised algorithm.

Min Cut — Monte Carlo Algorithm

Algorithm 1: IntuitiveKarger(G)

while *there are more than 2 supernodes*: **do**
 | Pick an edge $(u, v) \in E(G)$ uniformly at random;
 | Merge u and v ;
Output edges between the remaining two supernodes

Min Cut — Monte Carlo Algorithm

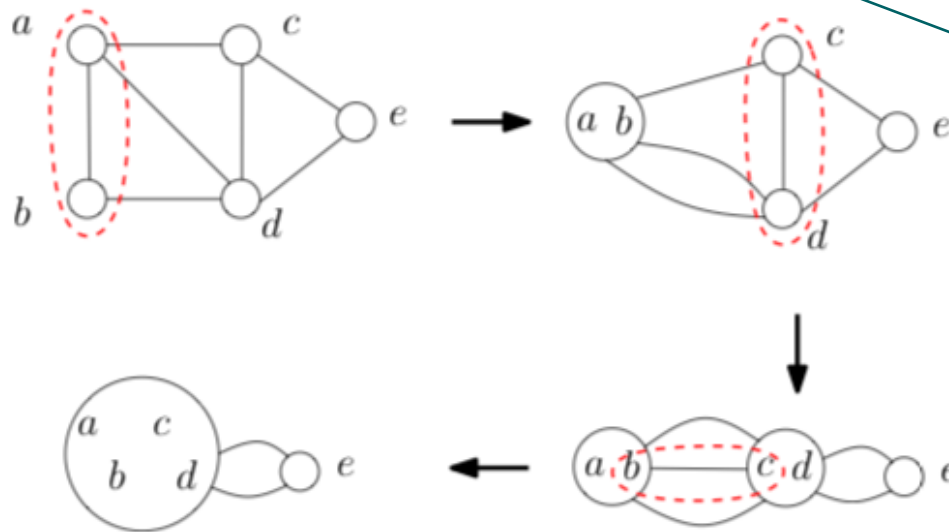
Algorithm 1: IntuitiveKarger(G)

while *there are more than 2 supernodes*: **do**

 Pick an edge $(u, v) \in E(G)$ uniformly at random;

 Merge u and v ;

Output edges between the remaining two supernodes



edge contraction

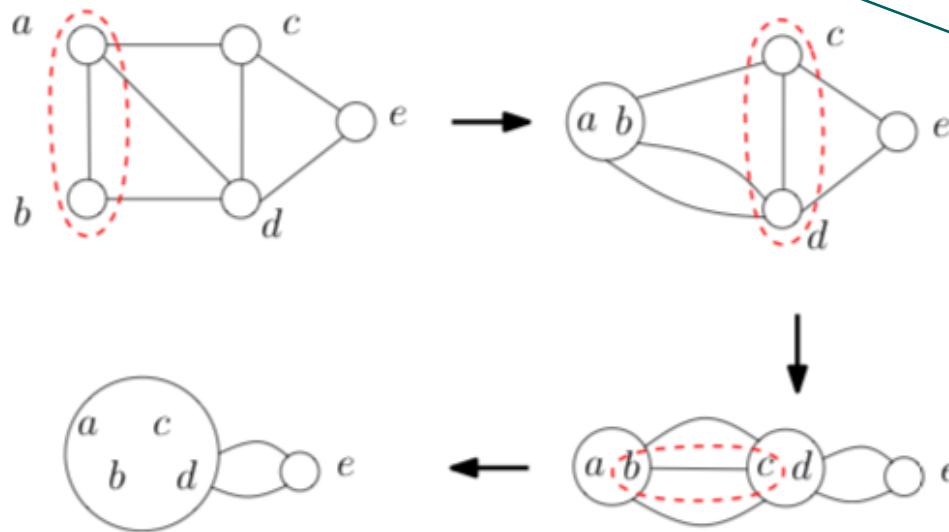
Min Cut — Monte Carlo Algorithm

Algorithm 1: IntuitiveKarger(G)

while *there are more than 2 supernodes*: **do**

 Pick an edge $(u, v) \in E(G)$ uniformly at random;
 Merge u and v ;

Output edges between the remaining two supernodes



edge contraction

The complexity is $\mathcal{O}(n^2)$ (every contraction takes $\mathcal{O}(n)$)

Min Cut — Monte Carlo Algorithm

What is the probability that the algorithm returns a correct result?

Min Cut — Monte Carlo Algorithm

What is the probability that the algorithm returns a correct result?

- let $e_1, e_2, \dots, e_{n-2}$ be the edges contracted by the algorithm
- fix a minimal cut S^* — it has k edges
- if edge e_i is across the cut (event E_i), then S^* is not the output of the algorithm

$$Pr\{S^* \text{ is the output}\} = Pr\{\bar{E}_1 \cap \bar{E}_2 \cap \dots \cap \bar{E}_{n-2}\}$$

$$Pr\{S^* \text{ is the output}\} = Pr\{\bar{E}_1\} \cdot Pr\{\bar{E}_2 | \bar{E}_1\} \cdot \dots \cdot Pr\{\bar{E}_{n-2} | \bar{E}_1 \cap \dots \cap \bar{E}_{n-3}\}$$

Min Cut — Monte Carlo Algorithm

What is the probability that the algorithm returns a correct result?

- let $e_1, e_2, \dots, e_{n-2}$ be the edges contracted by the algorithm
- fix a minimal cut S^* — it has k edges
- if edge e_i is across the cut (event E_i), then S^* is not the output of the algorithm

$$Pr\{S^* \text{ is the output}\} = Pr\{\bar{E}_1 \cap \bar{E}_2 \cap \dots \cap \bar{E}_{n-2}\}$$

$$Pr\{S^* \text{ is the output}\} = Pr\{\bar{E}_1\} \cdot Pr\{\bar{E}_2 | \bar{E}_1\} \cdot \dots \cdot Pr\{\bar{E}_{n-2} | \bar{E}_1 \cap \dots \cap \bar{E}_{n-3}\}$$

$Pr\{E_1\} = k/m$ where m is the total number of edges

- since the minimal cut has k edges, the out-degree (deg) of all vertices is at least k
- as $\sum_{v \in V} \deg(v) = 2m$, we get $2m \geq kn$ and thus $Pr\{E_1\} = k/m < 2/n$
- $Pr\{\bar{E}_1\} \geq 1 - 2/n$

Min Cut — Monte Carlo Algorithm

What is the probability that the algorithm returns a correct result?

- let $e_1, e_2, \dots, e_{n-2}$ be the edges contracted by the algorithm
- fix a minimal cut S^* — it has k edges
- if edge e_i is across the cut (event E_i), then S^* is not the output of the algorithm

$$Pr\{S^* \text{ is the output}\} = Pr\{\bar{E}_1 \cap \bar{E}_2 \cap \dots \cap \bar{E}_{n-2}\}$$

$$Pr\{S^* \text{ is the output}\} = Pr\{\bar{E}_1\} \cdot Pr\{\bar{E}_2 | \bar{E}_1\} \cdot \dots \cdot Pr\{\bar{E}_{n-2} | \bar{E}_1 \cap \dots \cap \bar{E}_{n-3}\}$$

- after one merge we have $2m \geq k(n-1)$ and thus $Pr\{\bar{E}_2 | \bar{E}_1\} \geq 1 - 2/(n-1)$
- after two merges we have $2m \geq k(n-2)$ and thus

$$Pr\{\bar{E}_3 | \bar{E}_2 \cap \bar{E}_1\} \geq 1 - 2/(n-2)$$

Min Cut — Monte Carlo Algorithm

What is the probability that the algorithm returns a correct result?

- let $e_1, e_2, \dots, e_{n-2}$ be the edges contracted by the algorithm
- fix a minimal cut S^* — it has k edges
- if edge e_i is across the cut (event E_i), then S^* is not the output of the algorithm

$$Pr\{S^* \text{ is the output}\} = Pr\{\bar{E}_1 \cap \bar{E}_2 \cap \dots \cap \bar{E}_{n-2}\}$$

$$Pr\{S^* \text{ is the output}\} = Pr\{\bar{E}_1\} \cdot Pr\{\bar{E}_2 | \bar{E}_1\} \cdot \dots \cdot Pr\{\bar{E}_{n-2} | \bar{E}_1 \cap \dots \cap \bar{E}_{n-3}\}$$

- after one merge we have $2m \geq k(n-1)$ and thus $Pr\{\bar{E}_2 | \bar{E}_1\} \geq 1 - 2/(n-1)$
- after two merges we have $2m \geq k(n-2)$ and thus

$$Pr\{\bar{E}_3 | \bar{E}_2 \cap \bar{E}_1\} \geq 1 - 2/(n-2)$$

- etc.

Min Cut — Monte Carlo Algorithm

What is the probability that the algorithm returns a correct result?

- let $e_1, e_2, \dots, e_{n-2}$ be the edges contracted by the algorithm
- fix a minimal cut S^* — it has k edges
- if edge e_i is across the cut (event E_i), then S^* is not the output of the algorithm

$$Pr\{S^* \text{ is the output}\} = Pr\{\bar{E}_1 \cap \bar{E}_2 \cap \dots \cap \bar{E}_{n-2}\}$$

$$Pr\{S^* \text{ is the output}\} = Pr\{\bar{E}_1\} \cdot Pr\{\bar{E}_2 | \bar{E}_1\} \cdot \dots \cdot Pr\{\bar{E}_{n-2} | \bar{E}_1 \cap \dots \cap \bar{E}_{n-3}\}$$

$$\begin{aligned} &\geq \left(1 - \frac{2}{n}\right) \left(1 - \frac{2}{n-1}\right) \left(1 - \frac{2}{n-2}\right) \cdots \left(1 - \frac{2}{n-(n-4)}\right) \left(1 - \frac{2}{n-(n-3)}\right) \\ &= \left(\frac{n-2}{n}\right) \left(\frac{n-3}{n-1}\right) \left(\frac{n-4}{n-2}\right) \cdots \left(\frac{2}{4}\right) \left(\frac{1}{3}\right) = 2/n(n-1) \end{aligned}$$

The success probability $1/\Theta(n^2)$ is quite small

Min Cut — Monte Carlo Algorithm with Amplification

- let C be constant
- run the algorithm $(C \cdot n^2 \cdot \ln n)$ times
- the complexity is $\mathcal{O}(n^4 \log n)$
- the probability that the amplified algorithm return an incorrect result

$$\begin{aligned} &\leq \left(1 - \frac{2}{n(n-1)}\right)^{C \cdot \binom{n}{2} \cdot \ln n} \\ &\leq \exp\left(-\frac{2}{n(n-1)} C \binom{n}{2} \ln n\right) \quad (\text{Useful fact}). \text{ For any real } x, 1 + x \leq e^x \\ &= \exp(-C \ln n) \\ &= \frac{1}{n^C}. \end{aligned}$$

The success probability is $1 - 1/n^C$

practically 1, for large n and C

Min Cut — Improved Monte Carlo Algorithm [just main idea]

We can further improved the algorithm by the following observation:

- earlier merging operations are less risky — there is more edges and thus the probability that an edge across the cut is contracted is smaller

Min Cut — Improved Monte Carlo Algorithm [just main idea]

We can further improved the algorithm by the following observation:

- earlier merging operations are less risky — there is more edges and thus the probability that an edge across the cut is contracted is smaller

1 Step: Run the randomised algorithm

2 Step: Once the graph is small enough, run the brute-force algorithm in $\mathcal{O}(1)$

Min Cut — Improved Monte Carlo Algorithm [just main idea]

We can further improved the algorithm by the following observation:

- earlier merging operations are less risky — there is more edges and thus the probability that an edge across the cut is contracted is smaller

1 Step: Run the randomised algorithm

2 Step: Once the graph is small enough, run the brute-force algorithm in $\mathcal{O}(1)$

a common technique for advanced Monte Carlo algorithms

Min Cut — Improved Monte Carlo Algorithm [just main idea]

We can further improved the algorithm by the following observation:

- earlier merging operations are less risky — there is more edges and thus the probability that an edge across the cut is contracted is smaller

1 Step: Run the randomised algorithm

2 Step: Once the graph is small enough, run the brute-force algorithm in $\mathcal{O}(1)$

a common technique for advanced Monte Carlo algorithms

+ advanced amplification — random merging operations are repeated later

Min Cut — Improved Monte Carlo Algorithm [just main idea]

We can further improved the algorithm by the following observation:

- earlier merging operations are less risky — there is more edges and thus the probability that an edge across the cut is contracted is smaller

1 Step: Run the randomised algorithm

2 Step: Once the graph is small enough, run the brute-force algorithm in $\mathcal{O}(1)$

a common technique for advanced Monte Carlo algorithms

+ advanced amplification — random merging operations are repeated later

The resulting algorithm has complexity $\mathcal{O}(n^2 \log^2 n)$

- similar probability that the result is correct

Famous randomised algorithms for hard problems

Miller–Rabin primality test (Monte Carlo algorithm)

- advanced algebra + randomisation
- existing and practically very useful implementations

Polynomial identity testing

- decide if two given polynomial functions are identical

Several important graph algorithms

- perfect matching in bipartite graphs, graph isomorphism, etc.

Famous randomised algorithms for hard problems

Miller–Rabin primality test (Monte Carlo algorithm)

- advanced algebra + randomisation
- existing and practically very useful implementations

Polynomial identity testing

- decide if two given polynomial functions are identical

Several important graph algorithms

- perfect matching in bipartite graphs, graph isomorphism, etc.

Randomisation is not a silver bullet

Famous randomised algorithms for hard problems

Miller–Rabin primality test (Monte Carlo algorithm)

- advanced algebra + randomisation
- existing and practically very useful implementations

Polynomial identity testing

- decide if two given polynomial functions are identical

Several important graph algorithms

- perfect matching in bipartite graphs, graph isomorphism, etc.

Randomisation is not a silver bullet

- Can we effectively randomise in the SAT (boolean satisfiability) problem?

Famous randomised algorithms for hard problems

Miller–Rabin primality test (Monte Carlo algorithm)

- advanced algebra + randomisation
- existing and practically very useful implementations

Polynomial identity testing

- decide if two given polynomial functions are identical

Several important graph algorithms

- perfect matching in bipartite graphs, graph isomorphism, etc.

Randomisation is not a silver bullet

- Can we effectively randomise in the SAT (boolean satisfiability) problem?
- It seems we cannot — consider the situation with a single satisfiable valuation

Famous randomised algorithms for hard problems

Miller–Rabin primality test (Monte Carlo algorithm)

- advanced algebra + randomisation
- existing and practically very useful implementations

Polynomial identity testing

- decide if two given polynomial functions are identical

Several important graph algorithms

- perfect matching in bipartite graphs, graph isomorphism, etc.

Randomisation is not a silver bullet

- Can we effectively randomise in the SAT (boolean satisfiability) problem?
- It seems we cannot — consider the situation with a single satisfiable valuation

More about randomised algorithms at FIT — Complexity course
